

PART I

IMAGE PROCESSING

INSTRUCTOR: DR. NGUYEN NGOC TRUONG MINH

SCHOOL OF ELECTRICAL ENGINEERING, INTERNATIONAL UNIVERSITY (VNU-HCMC)

LECTURE VII – IMAGE RESTORATION

INSTRUCTOR: DR. NGUYEN NGOC TRUONG MINH

SCHOOL OF ELECTRICAL ENGINEERING, INTERNATIONAL UNIVERSITY (VNU-HCMC)

Ho Chi Minh City, Fall 2025

LECTURE CONTENT

- What is **noise** (in the context of image processing) and how can it be modeled?
- What are the **main types of noise** that may affect an image?
- What is **blurring** (in the context of image processing) and how can it be modeled?
- Which **noise removal techniques** are typically used in image processing?
- Which **deblurring techniques** are typically used in image processing?
- Chapter Summary – What have we learned?
- Problems

7.1 MODELING OF THE IMAGE DEGRADATION AND RESTORATION PROBLEM

- This lecture presents techniques used to *improve the appearance of an image* that has been subject to degradation and noise.
- It is assumed that an original image $f(x, y)$ has been subject to *some sort of quality degradation* (e.g., blurring caused by lack of focus or camera motion, atmospheric disturbances, or geometric distortions caused by imperfect lenses) that can be modeled by a function $h(x, y)$.
- The image may (also) have been contaminated by *additive noise* ($n(x, y)$). The resulting degraded image, $g(x, y)$, is the input for the image restoration algorithms.

7.1 MODELING OF THE IMAGE DEGRADATION AND RESTORATION PROBLEM

- These algorithms are usually implemented as restoration filters that are able to undo—to some extent—the degradation process, resulting in a restored image $r(x, y)$ (which should be interpreted as an estimate of the original image $f(x, y)$ and is often referred to as $\hat{f}(x, y)$).
- In other words, *the goal of restoration techniques is to obtain an image that is as close to the original image as possible.*

7.1 MODELING OF THE IMAGE DEGRADATION AND RESTORATION PROBLEM

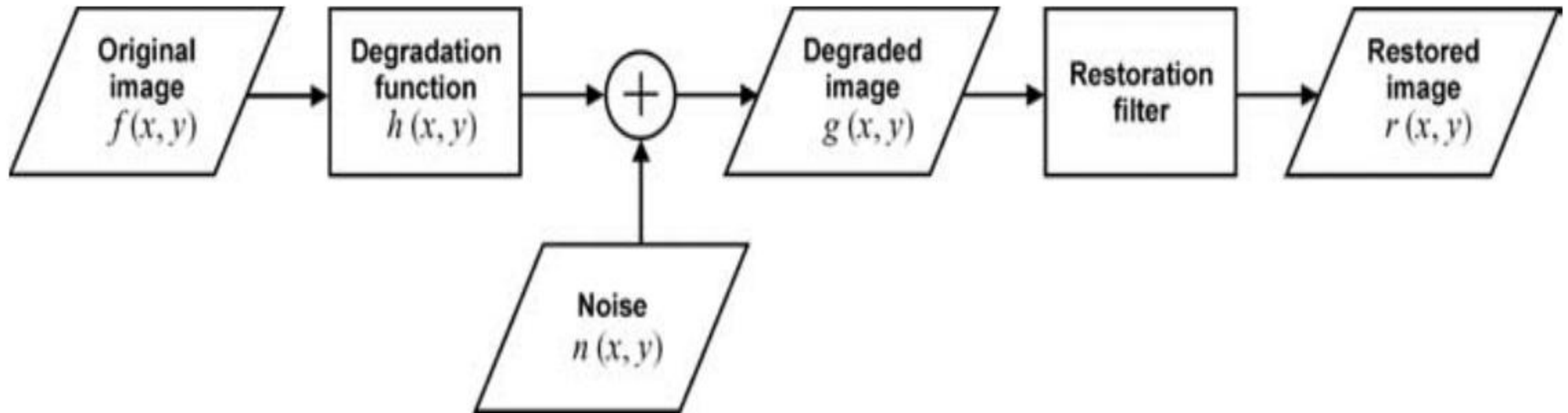


FIGURE 12.1 Image degradation and restoration.

7.1 MODELING OF THE IMAGE DEGRADATION AND RESTORATION PROBLEM

- Mathematically, the degradation and restoration problem can be described as

$$g(x, y) = f(x, y) * h(x, y) + n(x, y)$$

where $*$ denotes convolution.

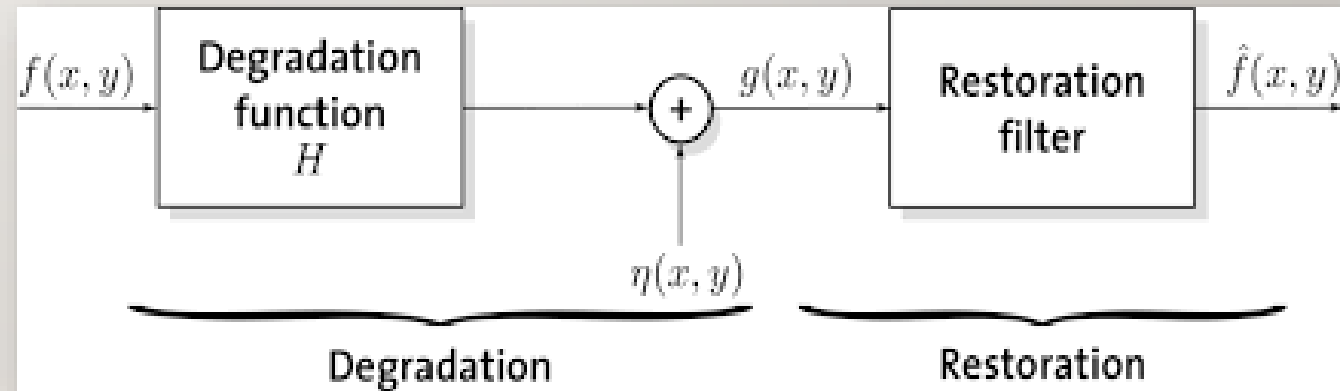
- From the convolution theorem, the following frequency-domain relation among the Fourier transform of $f(x, y)$, $g(x, y)$, $h(x, y)$, and $n(x, y)$ holds:

$$G(u, v) = F(u, v)H(u, v) + N(u, v)$$

- The *restoration filter* block in Figure 12.1 is typically designed following these steps:
 1. Collect *knowledge* about the degradation process (usually through examples of degraded images and knowledge of the image acquisition process).
 2. Use that knowledge to develop *a degradation model*.
 3. Develop the *inverse* degradation process and model it as a filter.

7.1 MODELING OF THE IMAGE DEGRADATION AND RESTORATION PROBLEM

- Restoration filters are, therefore, specifically designed to solve one particular type of degradation. They can work in the spatial domain or in the frequency domain.
- Note that the goal of image restoration techniques is clearly distinct from the goal of image enhancement techniques—such as the ones described in Lectures 5 and 6—where no mathematical modeling of (the inverse of) a degradation process is needed.



7.2 NOISE AND NOISE MODELS

- **Noise** can be defined as *any undesired artifact that contaminates an image*. The presence of noise in an image can be due to several sources, resulting in different types of noise, from thermal noise in acquisition devices to periodic noise in the communication channel used to transmit an image from a remote sensing location to a base station, among many others.



7.2 NOISE AND NOISE MODELS

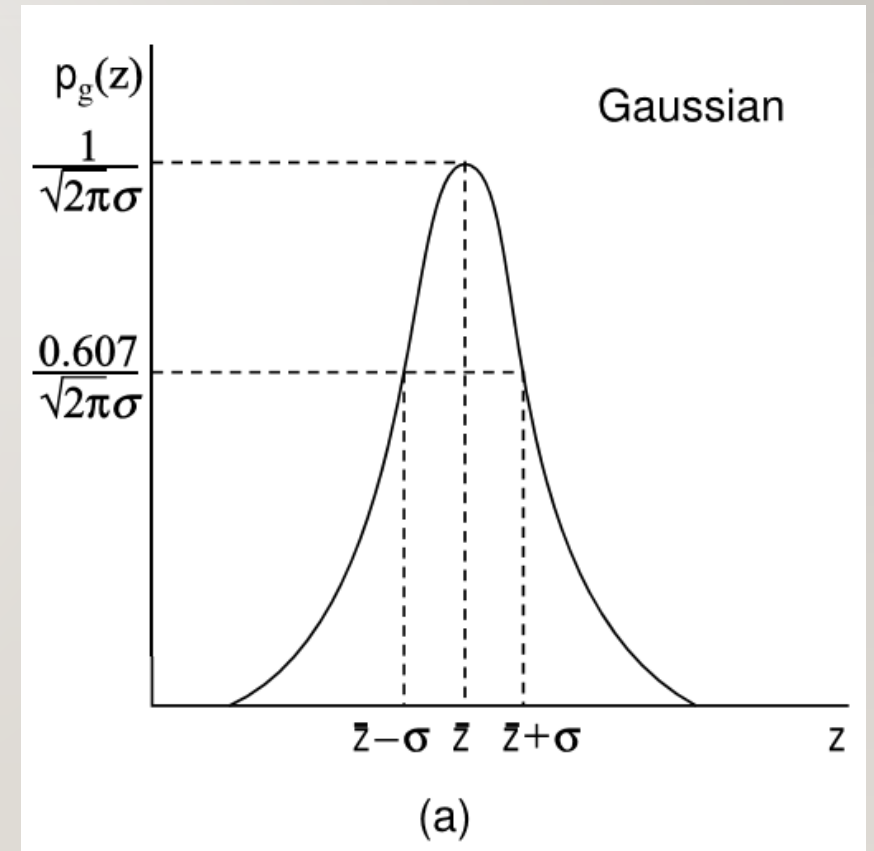
- In this section, we present an overview of the main types of noise that may be found in degraded digital images. Our main goals are threefold: (1) *to describe the main noise models in a mathematical and graphical way*, (2) *to remind that different types of noise will require different noise reduction techniques*, and (3) *to introduce the problem of noise estimation*.
- We treat noise as *a random variable* whose *probability density function (PDF)*, or histogram, describes its shape and distribution across the range of gray levels. In addition to that spatial-domain representation, noise patterns can sometimes also be represented in the frequency domain (through their frequency spectrum).

7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- **Gaussian Noise** The PDF of a Gaussian random variable z is given by

$$p_g(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\bar{z})^2/2\sigma^2}$$

where z represents the gray level, $\bar{z}$ is its mean, and σ is its standard deviation (σ^2 is called *the variance* of z).

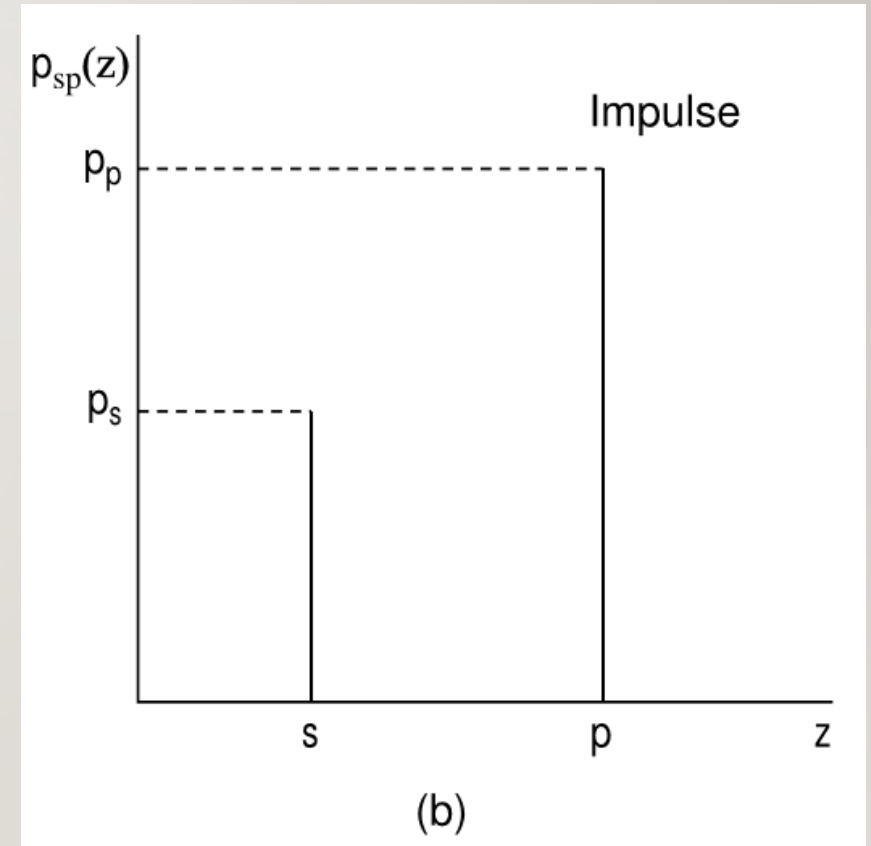


7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- *Impulse (Salt and Pepper)* Noise The PDF of (bipolar) impulse noise is given by

$$p_{sp}(z) = \begin{cases} P_p & \text{for } z = p \\ P_s & \text{for } z = s \\ 0 & \text{otherwise} \end{cases}$$

where P_p and P_s are the probability of occurrence of pixel whose values are equal to p (pepper) or s (salt), respectively.



7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- **Uniform Noise** The histogram of uniform noise is given by

$$p_u(z) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

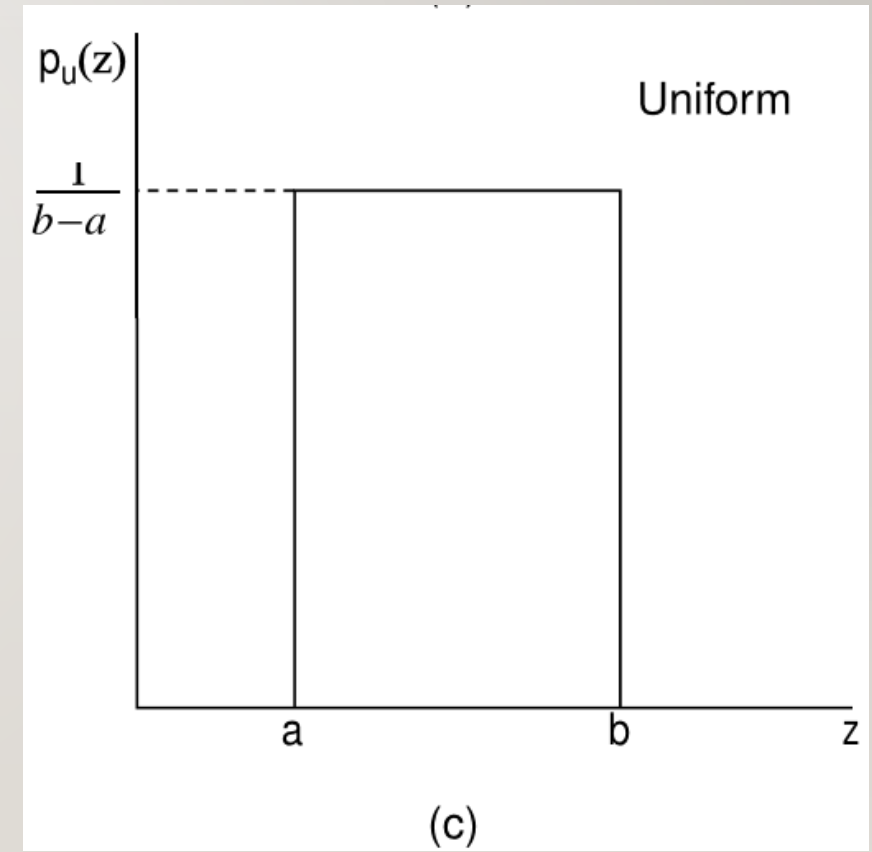
where $a \geq 0$ and $0 < a < b$.

The mean of the uniform noise is given by

$$\bar{z} = \frac{a+b}{2}$$

whereas the variance is given by

$$\sigma^2 = \frac{(b-a)^2}{12}$$



7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- **Rayleigh Noise** The PDF of Rayleigh noise is given by

$$p_r(z) = \begin{cases} \frac{2}{b}(z - a)e^{-(z-a)^2/b} & \text{for } z \geq a \\ 0 & \text{for } z < a \end{cases}$$

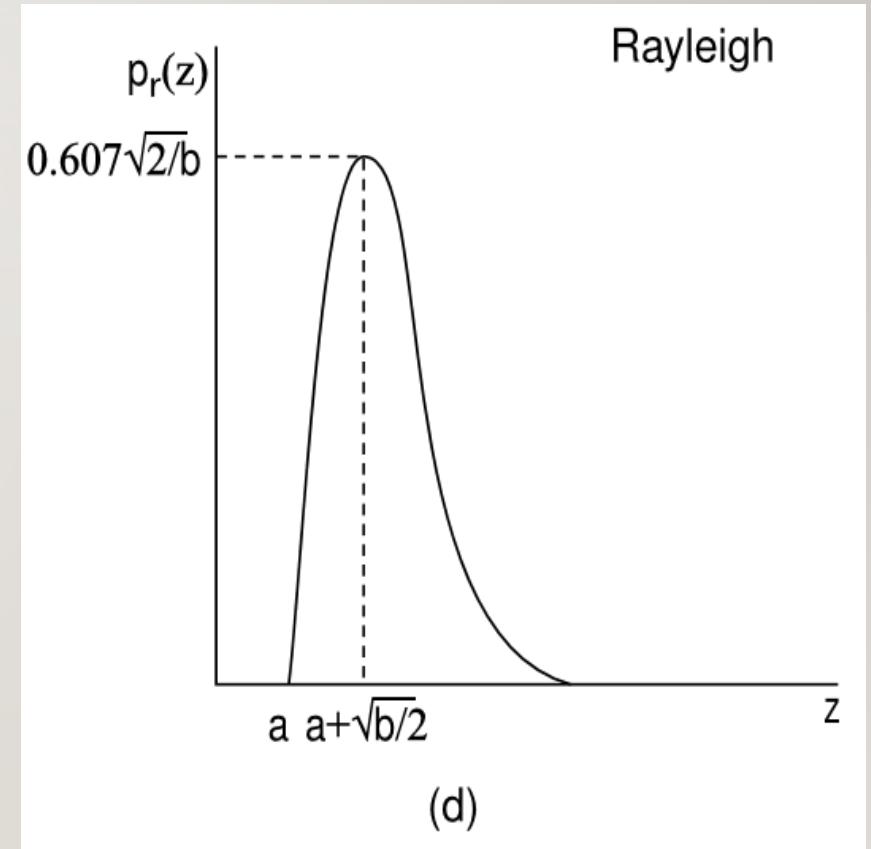
where $a \geq 0$ and $0 < a < b$.

The mean and the variance of the Rayleigh PDF are given by

$$\bar{z} = a + \sqrt{\pi b/4}$$

and

$$\sigma^2 = \frac{b(4 - \pi)}{4}$$



7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- **Gamma (Erlang) Noise** The histogram of gamma (Erlang) noise is given by

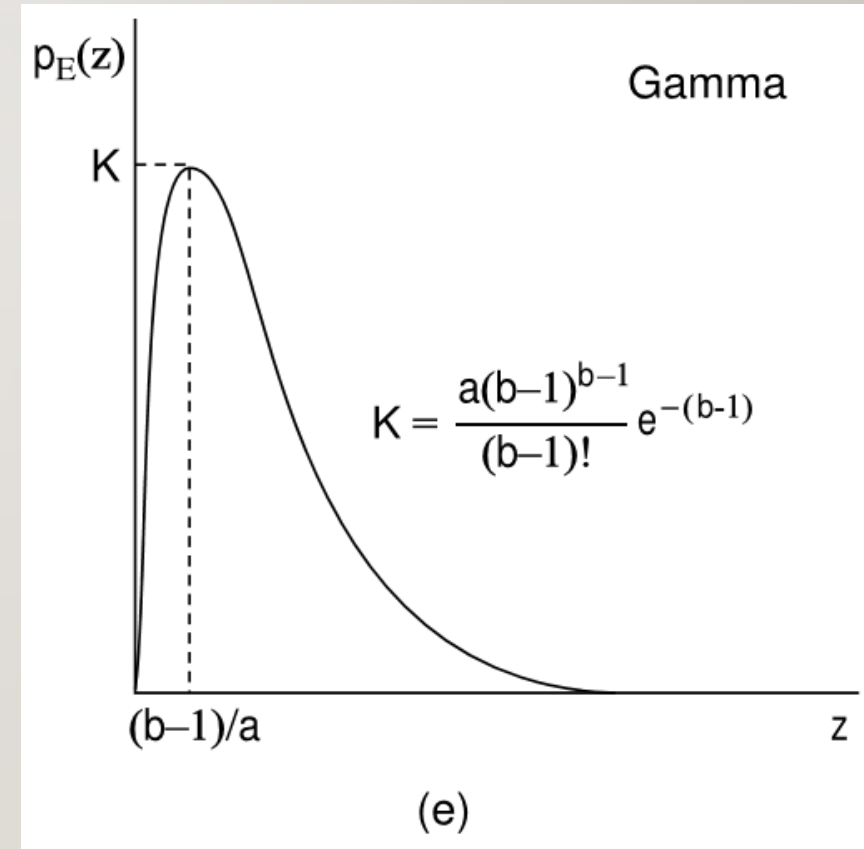
$$p_E(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

where $a > 0$, b is a positive integer, and “!” indicates factorial. The mean and the variance of the Rayleigh PDF are given by

$$\bar{z} = \frac{b}{a}$$

and

$$\sigma^2 = \frac{b}{a^2}$$



7.2.1 SELECTED NOISE PROBABILITY DENSITY FUNCTIONS

- **Exponential Noise** The PDF of exponential noise (a special case of the Erlang PDF, with $b = 1$) is given by

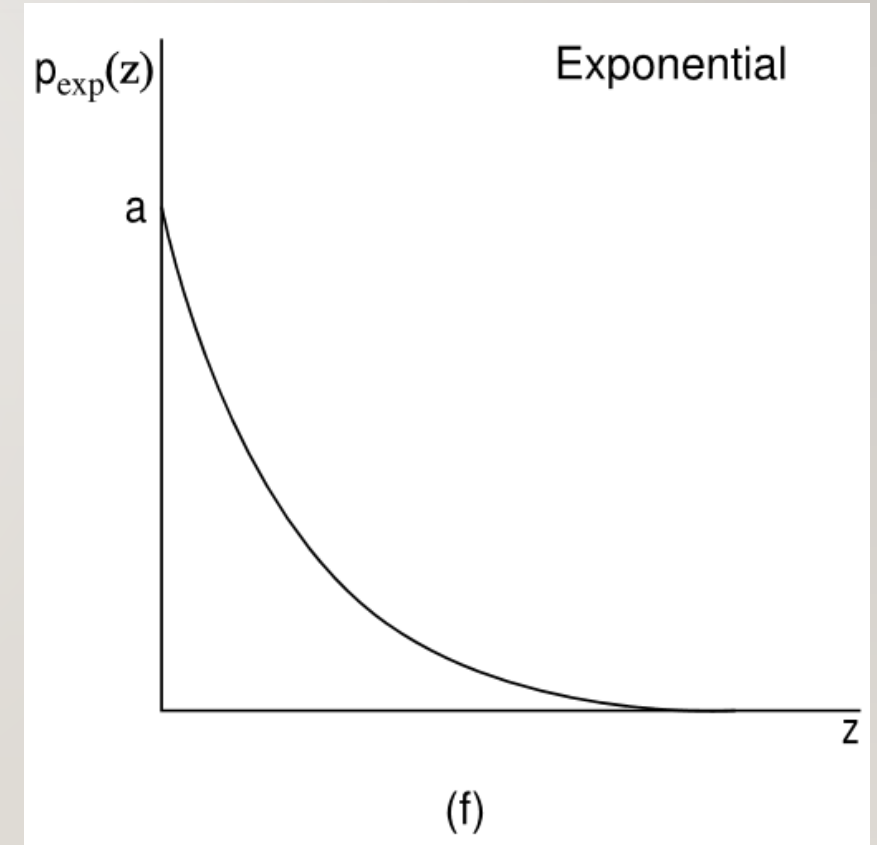
$$p_{exp}(z) = \begin{cases} ae^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

The mean and the variance of the exponential PDF are given by

$$\bar{z} = \frac{1}{a}$$

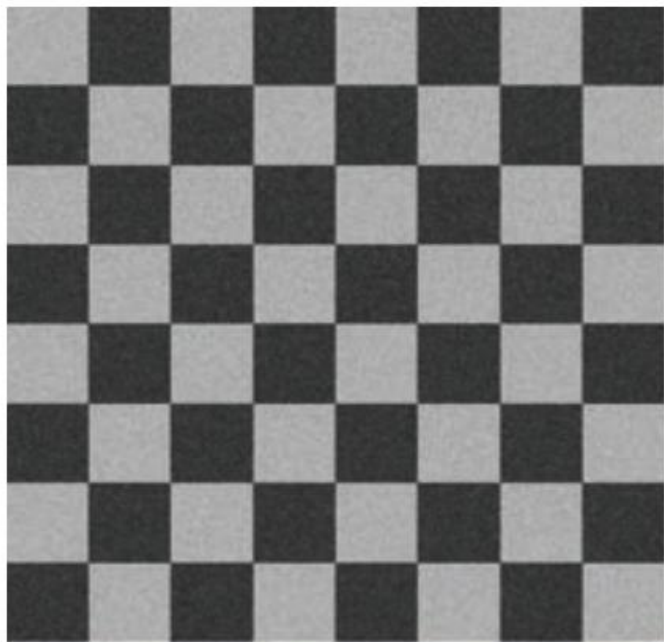
and

$$\sigma^2 = \frac{1}{a^2}$$

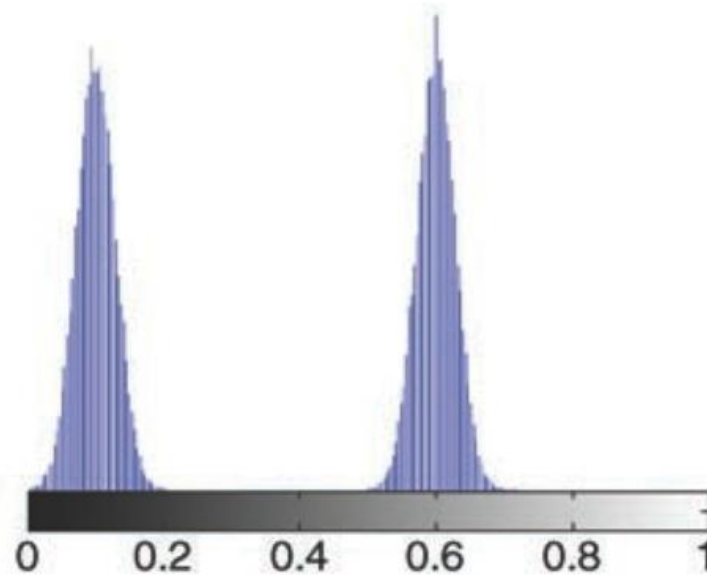


EXAMPLE 7.1

- Figures 12.3 and 12.4 show a test image contaminated by different types of noise and the resulting histograms. Since the test image contains only two gray levels, it is possible to map the histograms to the plots in Figure 12.2.



(a)

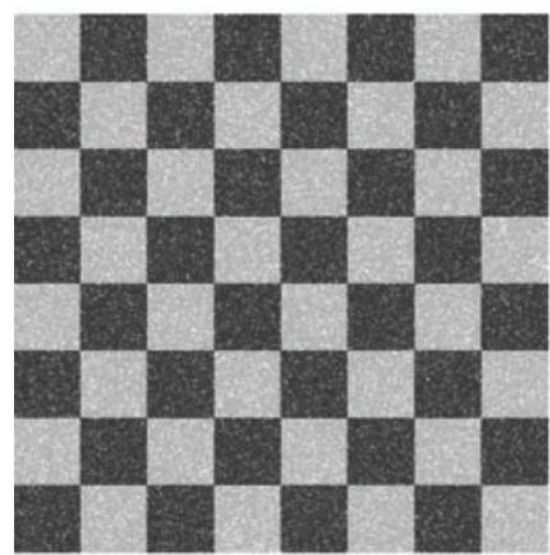


(b)

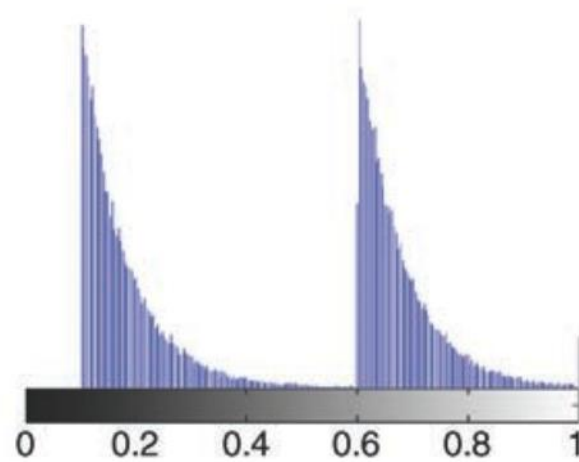
Test images and corresponding histograms for different types of noise: (a and b) Gaussian

EXAMPLE 7.1

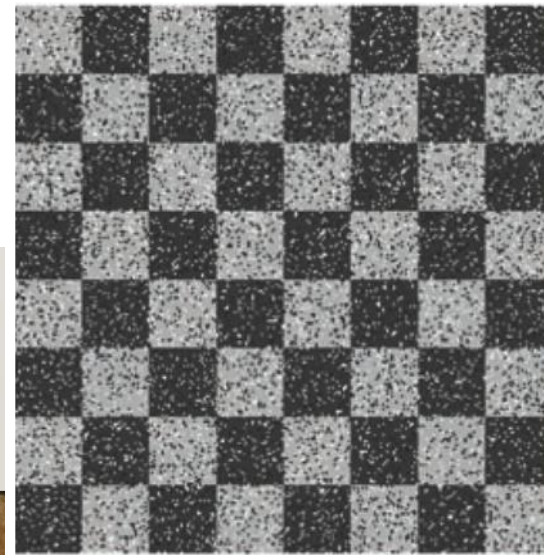
Test images and corresponding histograms for different types of noise: (c and d) exponential; (e and f) salt and pepper.



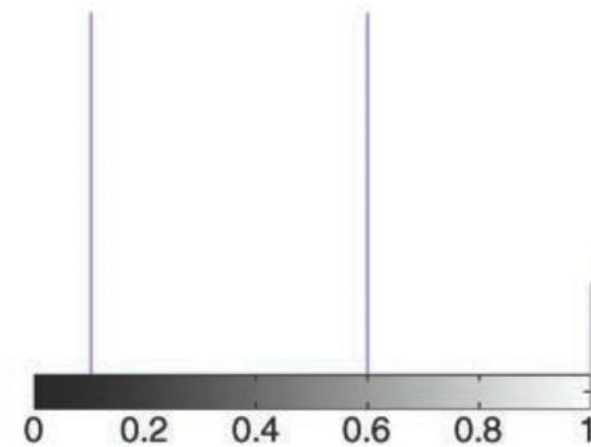
(c)



(d)

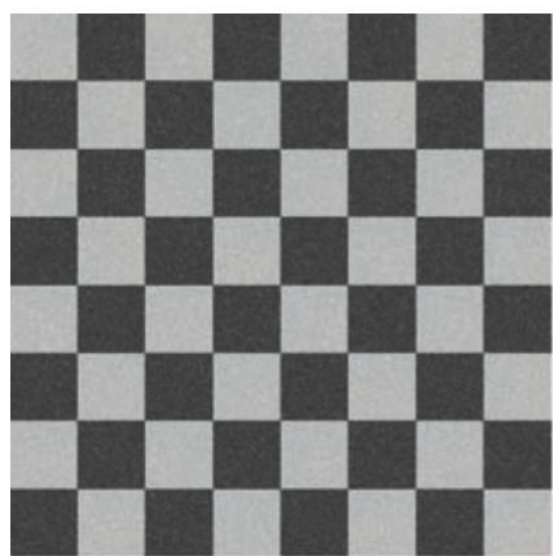


(e)

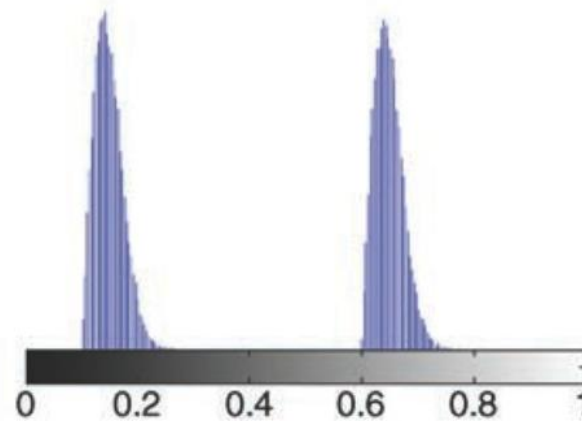


(f)

EXAMPLE 7.1

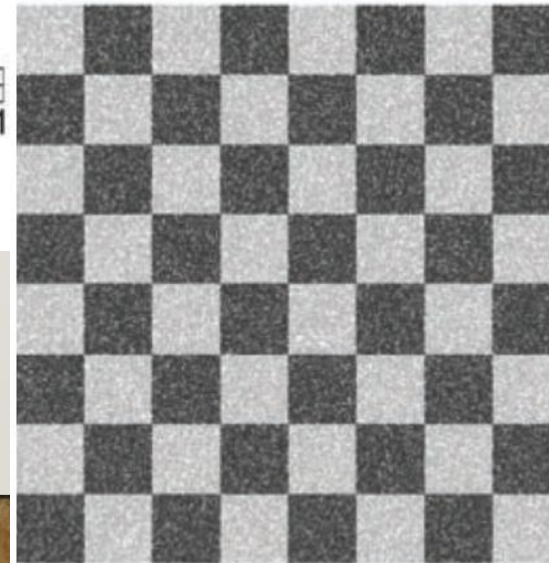


(a)

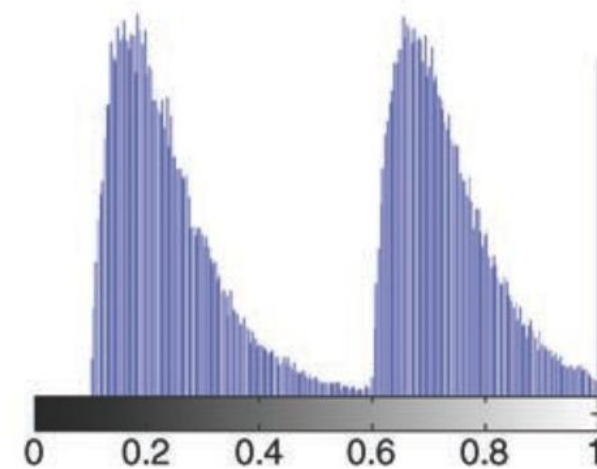


(b)

Test images and corresponding histograms for different types of noise: (a and b) Rayleigh; (c and d) Gamma.

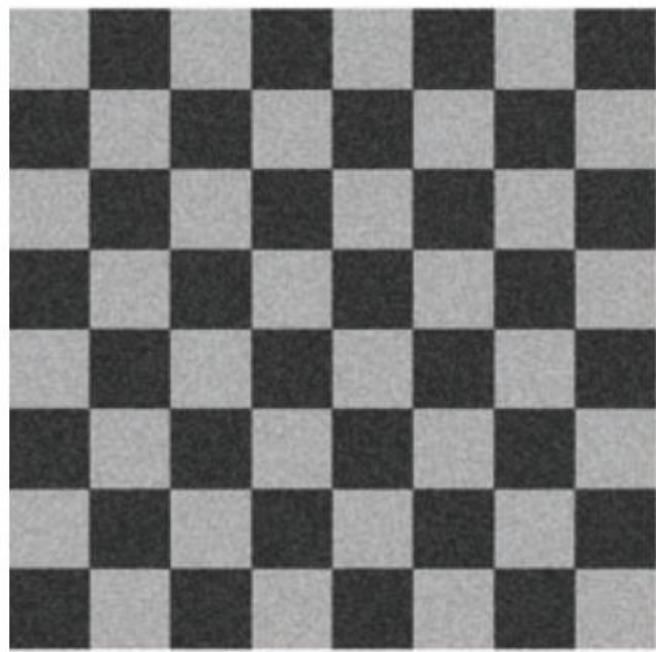


(c)

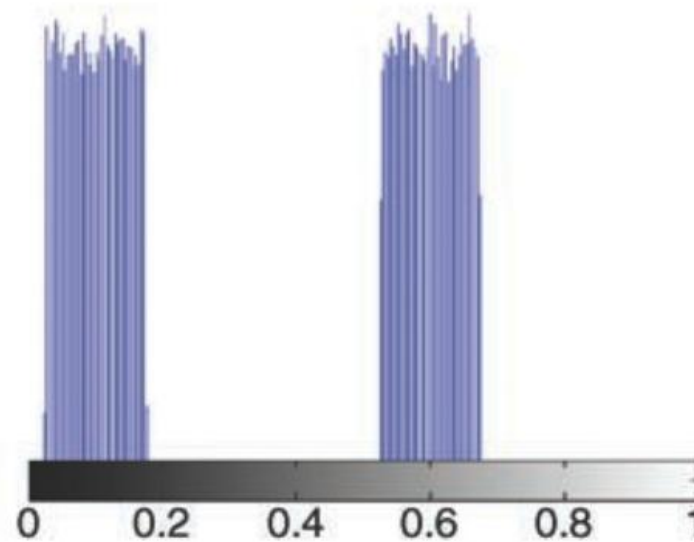


(d)

EXAMPLE 7.1



(e)



(f)

Test images and corresponding histograms for different types of noise: (e and f) uniform.

7.2.2 NOISE ESTIMATION

- It is often necessary to *estimate the type of noise* (and the main parameters of its PDF) before designing a solution for noise reduction. In cases where the acquisition device (sensor) is the primary source of noise, it is common to generate test images consisting of large patches of homogeneous gray-level values and observe the resulting histograms.
- If creating such test images is not possible, *a widely used alternative is to crop a relatively large homogeneous region* from an image and inspect its histogram: even though the histogram of the entire image may not provide an adequate hint as to the type of noise present in the image, the histogram of the cropped portion will.

EXAMPLE 7.2

- Figure 12.5 shows the process of noise estimation. The addition of noise to the original image (a) causes dramatic changes to its histogram (from (c) to (d)), without, however, providing a reliable hint as to the type of noise present in the image (b). Inspecting the histogram (e) of the cropped portion of the image (indicated by a rectangle in (b)) allows us to conclude that the noise is of Gaussian type.

EXAMPLE 7.2

In MATLAB

- MATLAB's IPT has a built-in function to add noise to an image: *imnoise*. It allows the generation of noisy versions of input images. It supports several types of noise, including Gaussian and salt and pepper additive noise and speckle multiplicative noise. It does not, however, support other types of noise described in this section (e.g., Rayleigh, uniform, Erlang, and exponential).

EXAMPLE 7.2

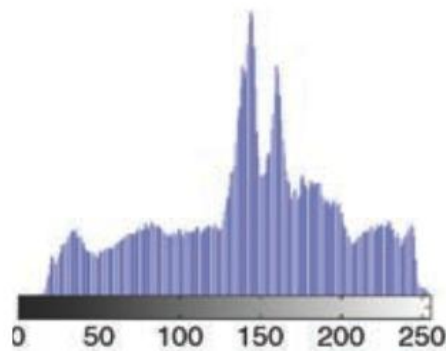
FIGURE 12.5 Estimating noise type from a homogeneous patch within an image: (a) original image; (b) noisy image (where the rectangle indicates a manually selected patch); (c) histogram of the original image; (d) histogram of the noisy image; (e) histogram of selected patch showing clearly that the noise is of Gaussian type in this case.



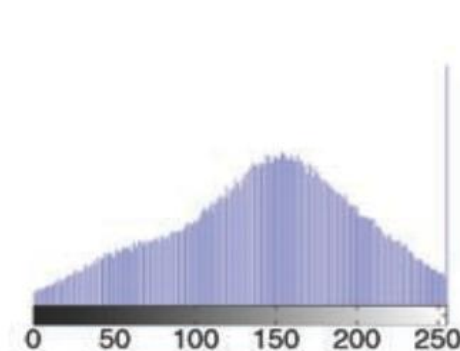
(a)



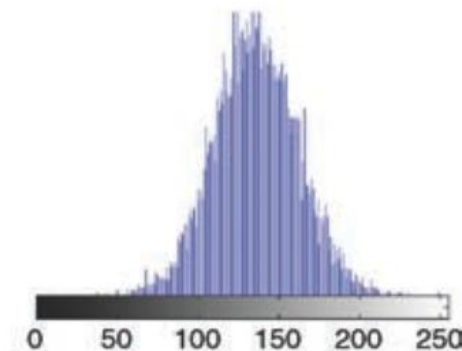
(b)



(c)



(d)



(e)

7.3 NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUE

- Noise reduction filters work under the assumption that the only degradation present in an image is additive noise.

$$g(x, y) = f(x, y) + n(x, y)$$

and

$$G(u, v) = F(u, v) + N(u, v)$$

- This section presents the most popular spatial-domain techniques for noise reduction. There are two main groups of spatial-domain noise reduction techniques: mean filters (Section 7.3.1) and order statistic (or rank) filters (Section 7.3.2).
- At the end of this section, we will briefly introduce the third category of spatial domain noise reduction techniques: adaptive filters.

7.3.1 MEAN FILTERS

- In this section, we revisit the arithmetic mean filter introduced in Section 10.3.1—from the perspective of its noise reduction capabilities—and expand the discussion to include other related filters and discuss their performance.

7.3.1 MEAN FILTERS

Arithmetic Mean Filter

The arithmetic mean filter, also known as *averaging filter*, operates on an $m \times n$ sliding window by calculating the average of all pixel values within the window and replacing the center pixel value in the destination image with the result. Its mathematical formulation:

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(r,c) \in W} g(r, c)$$

where g is the noisy image, $\hat{f}$ is the restored image, and r and c are the row and column coordinates, respectively, within a window W of size $m \times n$ where the operation takes place. The arithmetic mean filter causes a certain amount of blurring (proportional to the window size) to the image, thereby reducing the effects of noise. It can be used to reduce noise of different types, but *works best for Gaussian, uniform, or Erlang noise*.

7.3.1 MEAN FILTERS

Geometric Mean Filter

The geometric mean filter is a variation of the arithmetic mean filter and is primarily used on images with *Gaussian noise*. This filter is known to retain image detail better than the arithmetic mean filter. Its mathematical formulation is as follows:

$$\hat{f}(x, y) = \left[\prod_{(r,c) \in W} g(r, c) \right]^{1/mn}$$

7.3.1 MEAN FILTERS

Harmonic Mean Filter

The harmonic mean filter is yet another variation of the arithmetic mean filter and is useful for images with *Gaussian or salt noise*. Black pixels (pepper noise) are not filtered. The filter's mathematical formulation is as follows:

$$\hat{f}(x, y) = \frac{mn}{\sum_{(r,c) \in W} (1/g(r, c))}$$

7.3.1 MEAN FILTERS

Contraharmonic Mean Filter

The contra-harmonic mean filter is another variation of the arithmetic mean filter and is primarily used for *filtering salt or pepper noise* (but not both).

Images with salt noise can be filtered using negative values of R , whereas those with pepper noise can be filtered using positive values of R . The filter's mathematical formulation is:

$$\hat{f}(x, y) = \frac{\sum_{(r,c) \in W} g(r, c)^{R+1}}{\sum_{(r,c) \in W} g(r, c)^R}$$

where R is called the *order* of the filter.

EXAMPLE 7.3

- Figure 12.6 shows the effects of applying different filters to an image corrupted by Gaussian noise of zero mean and variance 0.01.
- The result obtained with any of the four filters contains less noise than that with the image in Figure 12.6b. Moreover, we may also have noticed that the geometric and harmonic mean filters generate more dark pixels in the resulting image.
- Finally, the figure also helps to confirm the trade-off involved when choosing the mask size for a mean averaging filter: the result for Figure 12.6d (5x5 window) contains less noise—but is significantly more blurry—than the one for Figure 12.6c (3x3 window).

EXAMPLE 7.3

FIGURE 12.6 (a) Original image; (b) image with Gaussian noise; (c) result of 3×3 arithmetic mean filtering;



(a)



(b)



(c)

EXAMPLE 7.3

FIGURE 12.6 (d) result of 5×5 arithmetic mean filtering; (e) result of 3×3 geometric mean filtering; (f) result of 3×3 harmonic mean filtering.



(d)



(e)



(f)

EXAMPLE 7.4

- Figure 12.7 shows the effects of applying different filters to an image corrupted by salt and pepper noise.
- The result obtained with the mean averaging filter is the only acceptable one. The geometric and the harmonic mean filters perform very poorly for the pepper portion of the noise, whereas the performance of the contraharmonic filter confirms what we had stated earlier: depending on the choice of R , it will reduce the amount of salt or pepper noise, but not both.

EXAMPLE 7.4

FIGURE 12.7 (a) Image with salt and pepper noise; (b) result of 3×3 arithmetic mean filtering; (c) result of 3×3 geometric mean filtering;



(a)



(b)



(c)

EXAMPLE 7.4

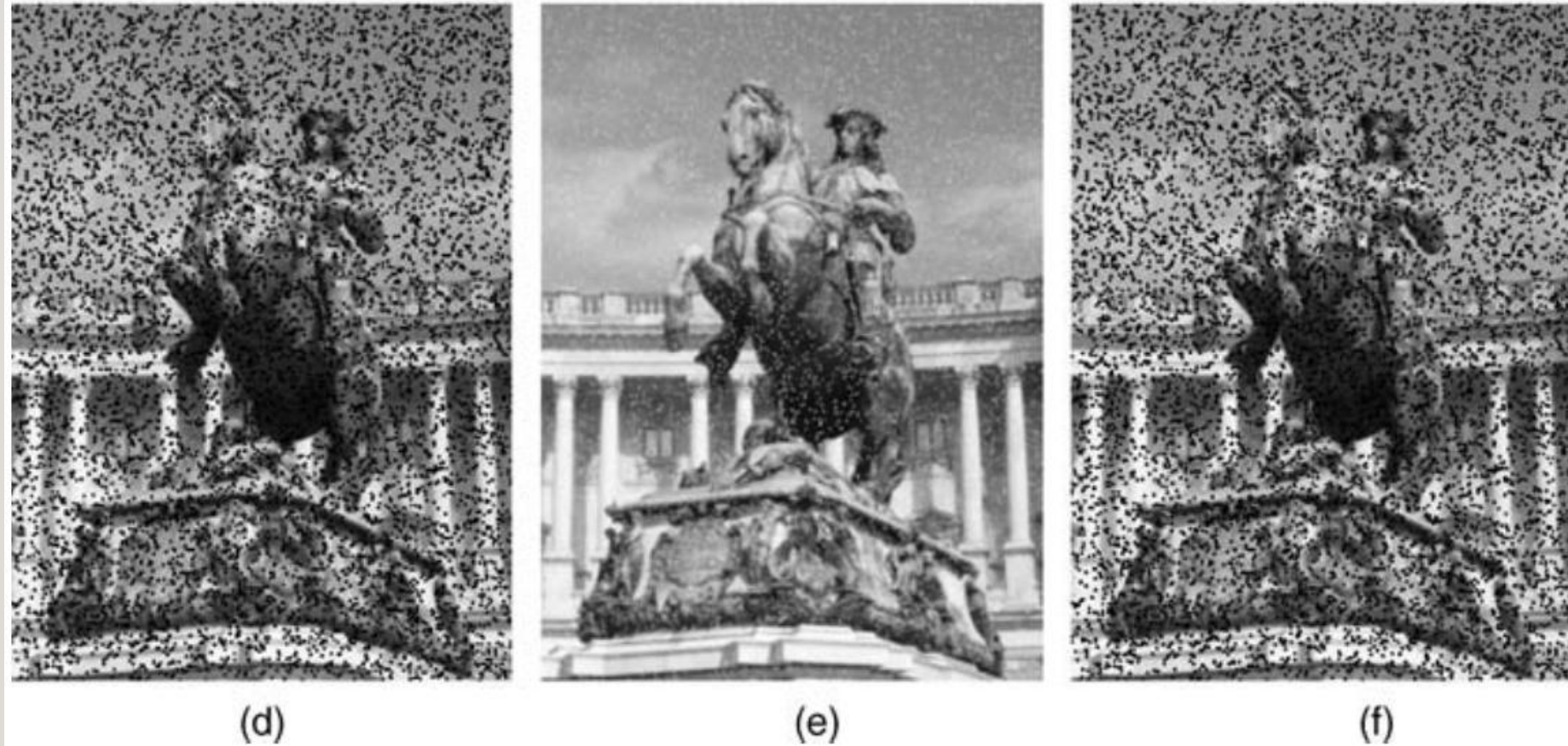


FIGURE 12.7 (d) result of 3×3 harmonic mean filtering; (e) result of 3×3 contraharmonic mean filtering with $R = 0.5$; (f) result of 3×3 contraharmonic mean filtering with $R = -0.5$.

7.3.2 ORDER STATISTIC FILTERS

- Order statistic filters (also known as *rank filters*, or simply *order filters*) operate on a neighborhood around a reference pixel by ordering (i.e., ranking) the pixel values and then performing an operation on those ordered values to obtain the new value for the reference pixel. Order statistic filters perform very well in the presence of salt and pepper noise but are computationally more expensive than mean filters.
- In this section, we revisit the median filter introduced in Section 10.3.4—from the perspective of its noise reduction capabilities—and expand the discussion to include other related filters (the min filter, the max filter, the midpoint filter, and the alpha trimmed mean filter) and discuss their performance.

7.3.2 ORDER STATISTIC FILTERS

The Median Filter

The most popular and useful of the rank filters is the median filter. It works by selecting the middle pixel value from the ordered set of values within the $m \times n$ neighborhood (W) around the reference pixel. If mn is an even number (which is not common), the arithmetic average of the two values closest to the middle of the ordered set is used instead. Mathematically,

$$\hat{f}(x, y) = \text{median}\{g(r, c) | (r, c) \in W\}$$

There have been many variants, extensions, and optimized implementations of the median filter proposed in the literature.



7.3.2 ORDER STATISTIC FILTERS

The Min and Max Filters

The *min* and *max* filters also work on a ranked set of pixel values. Contrary to the median filter—which replaces the reference pixel with the median of the ordered set—the *min* filter, also known as the zeroth percentile filter, replaces it with the lowest value instead. Mathematically,

$$\hat{f}(x, y) = \min\{g(r, c) | (r, c) \in W\}$$

Similarly, the *max* filter, also known as the 100th percentile filter, replaces the reference pixel within the window with the highest value, that is,

$$\hat{f}(x, y) = \max\{g(r, c) | (r, c) \in W\}$$

The min filter is useful for reduction of salt noise, whereas the max filter can help remove pepper noise.

7.3.2 ORDER STATISTIC FILTERS

The Midpoint Filter

The midpoint filter calculates the average of the highest and lowest pixel values within a window, thereby combining order statistics and averaging into one filter. It is used to reduce Gaussian and uniform noise in images.

Mathematically,

$$\hat{f}(x, y) = \frac{1}{2} [\max\{g(r, c) | (r, c) \in W\} + \min\{g(r, c) | (r, c) \in W\}]$$

7.3.2 ORDER STATISTIC FILTERS

The Alpha-Trimmed Mean Filter

The alpha-trimmed filter uses another combination of order statistics and averaging, in this case an average of the pixel values closest to the median, after the D lowest and D highest values in an ordered set have been excluded. The rationale behind this filter is to allow its user to control its behavior by specifying the parameter D . for $D = 0$, the filter behaves as a regular arithmetic mean filter; for $D = (mn-1)/2$, it is equivalent to the median filter. It is used in cases where the image is corrupted by more than one type of noise, for example, salt and pepper (where the median filter performs well) and Gaussian (where the arithmetic mean filter shows satisfactory performance).

$$\hat{f}(x, y) = \frac{1}{mn - 2D} \sum_{(r,c) \in W} g(r, c)$$

where D is the number of pixel values excluded at each end of the ordered set, which can range from 0 to $(mn-1)/2$.

EXAMPLE 7.5

- Figure 12.8 shows the effects of applying different rank filters to an image corrupted by salt and pepper noise. The results obtained with the median filter (part (c)) are clearly better than the ones obtained with the average filter of same window size (part (b)) as expected. The midpoint filter not only is ineffective in removing this type of noise, but also makes it worse by replacing the noisy pixels (and their immediate neighbors) with the average between the minimum—pepper—and maximum—salt—values.

EXAMPLE 7.5

In MATLAB

- Sliding window neighborhood operations are implemented in the IPT using one of these two functions: *nlfilter* or *colfilt*. Both functions accept a (user-defined) function as a parameter. Such function can perform linear (e.g., averaging) or nonlinear (e.g., median) operations on the pixels within a window. Both functions might take long to process results on an image with hundreds (or thousands) of pixels in each dimension; they both provide a progress bar indicator to inform to the user that the processing is taking place, but *colfilt* is considerably faster than *nlfilter*.
- For rank filters, the IPT function *ordfilt2* makes it very easy to create the *min*, *max*, and *median* filters, as demonstrated in Tutorial 12.1. Since the median filter is by far the most popular rank filter, the IPT has another function with a more familiar name, *medfilt2*, that implements it.

EXAMPLE 7.5

FIGURE 12.8 (a) Image with salt and pepper noise; (b) result of 3×3 arithmetic mean filtering (for comparison);



(a)



(b)

EXAMPLE 7.5

FIGURE 12.8 (c) result of 3×3 median filtering; (d) result of 3×3 midpoint filtering.



(c)



(d)

7.3.3 ADAPTIVE FILTERS

- The basic idea behind adaptive filters is to design the filter in such a way that its behavior changes depending on the pixel values of the neighborhood currently being processed.
- A classical use of adaptive filters is found under the category of edge-preserving smoothing filters: in this case, the goal is to apply a low-pass filter to an image in a selective way, minimizing the edge blurring effect that would be present if a standard LPF had been applied to the image. There are many variants of adaptive filters for noise reduction and image restoration in the literature.



7.4 NOISE REDUCTION USING FREQUENCY-DOMAIN TECHNIQUES

- In this section, we introduce three additional types of frequency-domain filters—bandpass, bandreject, and notch—whose main application is in the reduction or removal of periodic noise.



7.4.1 PERIODIC NOISE

- This is a type of noise that usually arises as a result of electrical or electromechanical interference during the image acquisition process.



EXAMPLE 7.6

- Figure 12.9 shows an example of an image corrupted by periodic noise, the noise component, and its Fourier spectrum. The bright dots in the spectrum indicate four pairs of impulse functions, each pair corresponding to a sinusoidal noise source.

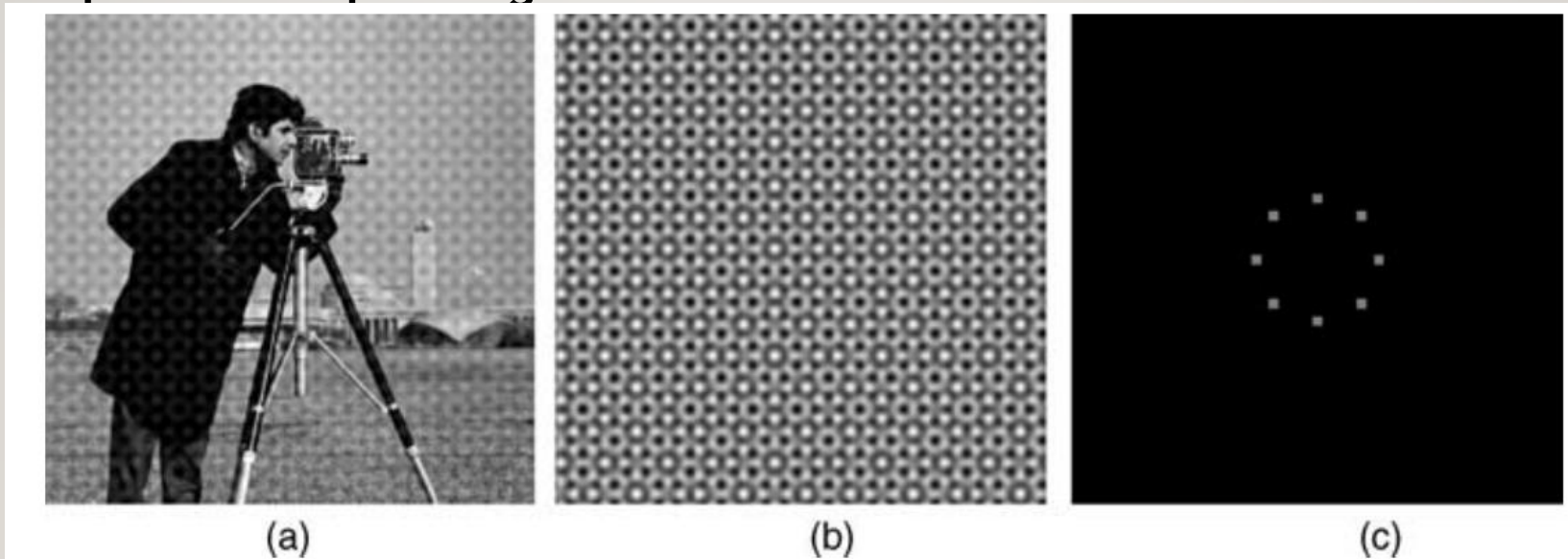


FIGURE 12.9 Example of an image corrupted by periodic noise: (a) noisy image; (b) periodic noise component; (c) the Fourier spectrum of the noise component (bright dots were enlarged for viewing purposes).

7.4.2 BAND-REJECT FILTER

- A bandreject filter, as its name suggests, attenuates frequency components within a certain range (the stopband of the filter), while leaving all other frequency components untouched (or amplifying them by a certain gain). The mathematical formulation for an ideal bandreject filter is as follows:

$$H_{br}^i(u, v) = \begin{cases} 1 & \text{if } D(u, v) < D_0 - \frac{W}{2} \\ 0 & \text{if } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 1 & \text{if } D(u, v) > D_0 + \frac{W}{2} \end{cases}$$

where $D(u, v)$ is the distance from the origin of the frequency spectrum, W is the width of the band, and D_0 is the radius of the circle-shaped band.

7.4.2 BAND-REJECT FILTER

- A *Butterworth* bandreject filter of order n is described mathematically as:

$$H_{br}^b(u, v) = \frac{1}{1 + \left[\frac{D(u, v)W}{D^2(u, v) - D_0^2} \right]^2}$$

whereas the transfer function for a *Gaussian* bandreject filter is given by the following equation:

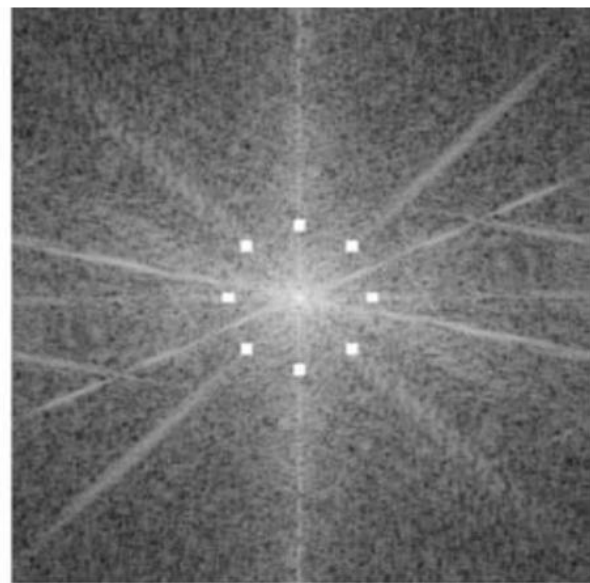
$$H_{br}^g(u, v) = 1 - e^{-\frac{1}{2} \left[\frac{D^2(u, v) - D_0^2}{D(u, v)W} \right]^2}$$

EXAMPLE 7.7

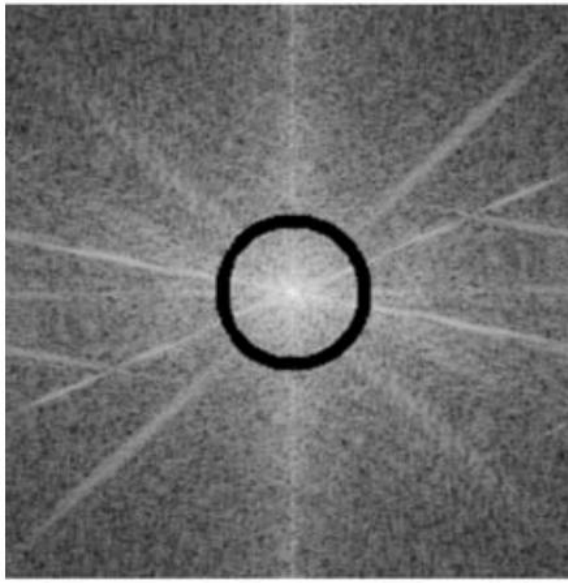
- Figure 12.10 shows an example of an ideal bandreject filter (of radius $D_0 = 32$ and width $W = 6$) used to reduce periodic noise. Although the periodic noise has been successfully removed, the resulting image shows an undesirable ringing effect as a result of the sharp transition between passband and stopband in the bandreject filter.
- FIGURE 12.10 Example of using a bandreject filter to reduce periodic noise: (a) noisy image; (b) noisy image spectrum (the eight spots corresponding to the noise have been made brighter and bigger for visualization purposes); (c) the Fourier spectrum of the image after applying the bandreject filter; (d) resulting image.*



(a)



(b)



(c)



(d)

7.4.3 BAND-PASS FILTER

- A bandpass filter allows certain frequencies (within its passband) to be preserved while attenuating all others. It is, in effect, the opposite of a bandreject filter (Section 12.4.2), and its mathematical formulation can be simply stated as follows:

$$H_{bp}(u, v) = 1 - H_{br}(u, v)$$

where $H_{br}(u, v)$ is the transfer function of a bandreject filter.



7.4.3 BAND-PASS FILTER

- *Ideal BPF*

$$H_{bp}^i(u, v) = \begin{cases} 0 & \text{if } D(u, v) < D_0 - \frac{W}{2} \\ 1 & \text{if } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 0 & \text{if } D(u, v) > D_0 + \frac{W}{2} \end{cases}$$

- *Butterworth BPF*

$$H_{bp}^b(u, v) = \frac{\left[\frac{D(u, v)W}{D^2(u, v) - D_0^2} \right]^{2n}}{1 + \left[\frac{D(u, v)W}{D^2(u, v) - D_0^2} \right]^{2n}}$$

- *Gaussian BPF*

$$H_{bp}^g(u, v) = e^{-\frac{1}{2} \left[\frac{D^2(u, v) - D_0^2}{D(u, v)W} \right]^2}$$

7.5 IMAGE DEBLURRING TECHNIQUES

- The goal of image deblurring techniques is to process an image that has been subject to blurring caused, for example, by camera motion during image capture or poor focusing of the lenses. The simplest image deblurring filtering technique, inverse filtering, operates in the frequency domain and assuming that there is no significant noise in the degraded image,

$$G(u, v) = F(u, v)H(u, v) + 0$$

which leads to

$$F(u, v) = \frac{G(u, v)}{H(u, v)} = G(u, v) \frac{1}{H(u, v)}$$

where the term $1/H(u, v)$ is the FT of the restoration filter, which will be denoted by $R_{inv}(u, v)$.

7.5 IMAGE DEBLURRING TECHNIQUES

- There are two common solutions to the former problem:
 1. Apply a low-pass filter with transfer function $L(u, v)$ to the division, thus limiting the restoration to a range of frequencies below *the restoration cutoff* frequency, that is,

$$F(u, v) = \frac{G(u, v)}{H(u, v)} L(u, v)$$

For frequencies within the filter's passband, the filter's gain is set to any desired positive value (usually the gain is set to 1); for frequencies within the filter's stopband, the gain should be zero. If the transition between the passband and stopband is too sharp (as in the case of the ideal LPF), however, ringing artifacts may appear in the restored image; other filter types (e.g., Butterworth with high order) may be preferred in this case.

7.5 IMAGE DEBLURRING TECHNIQUES

2. Use constrained division where a threshold value T is chosen such that if $|H(u, v)| < T$, the division does not take place and the original value is kept untouched:

$$F(u, v) = \begin{cases} \frac{G(u, v)}{H(u, v)} & \text{if } |H(u, v)| \geq T \\ G(u, v) & \text{otherwise} \end{cases}$$

EXAMPLE 7.8

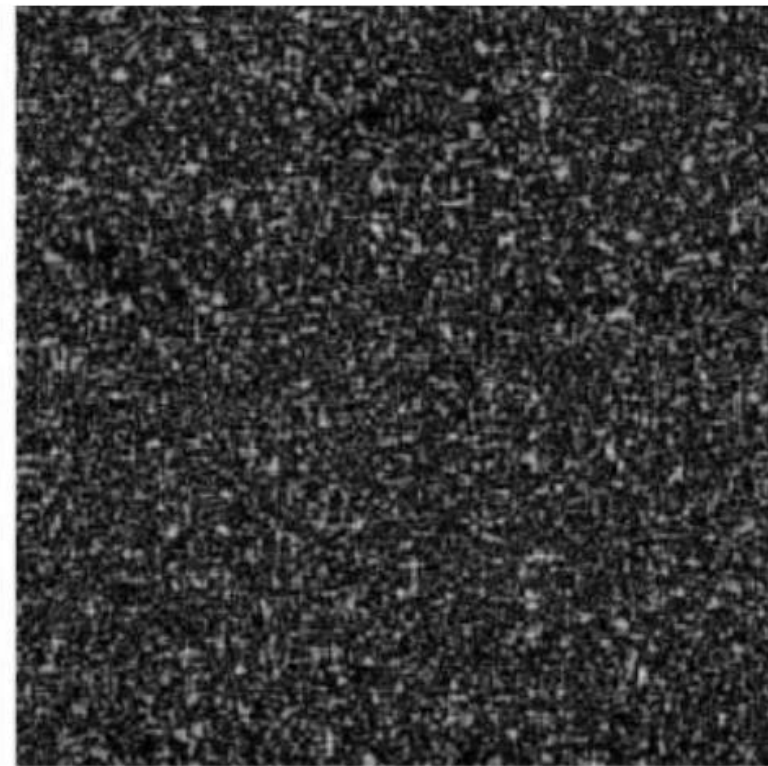
- Figure 12.11 shows an example of image restoration using inverse filtering. Part (a) shows the input (blurry) image. Part (b) shows the result of naively applying inverse filtering (equation (12.42)), which is completely unacceptable due to the division by very small values of $H(u, v)$. Parts (c) and (d) show the results of applying a 10th-order Butterworth low-pass filter to the division, with different cutoff frequencies. Parts (e) and (f) show the results of using constrained division, with different values for the threshold T .
- Motion deblurring can be considered a special case of inverse filtering. Figure 12.12 shows the results of applying inverse filtering with constrained division to a blurry image (generated using *fspecial('motion',10,0)* to simulate a horizontal displacement equivalent to 10 pixels).

EXAMPLE 7.8

- FIGURE 12.11 Example of image restoration using inverse filtering: (a) input (blurry) image; (b) result of naive inverse filtering;



(a)



(b)

EXAMPLE 7.8

- FIGURE 12.11 Example of image restoration using inverse filtering: (c) applying a 10th-order Butterworth low-pass filter with cutoff frequency of 20 to the division; (d) same as (c), but with cutoff frequency of 50;



(c)



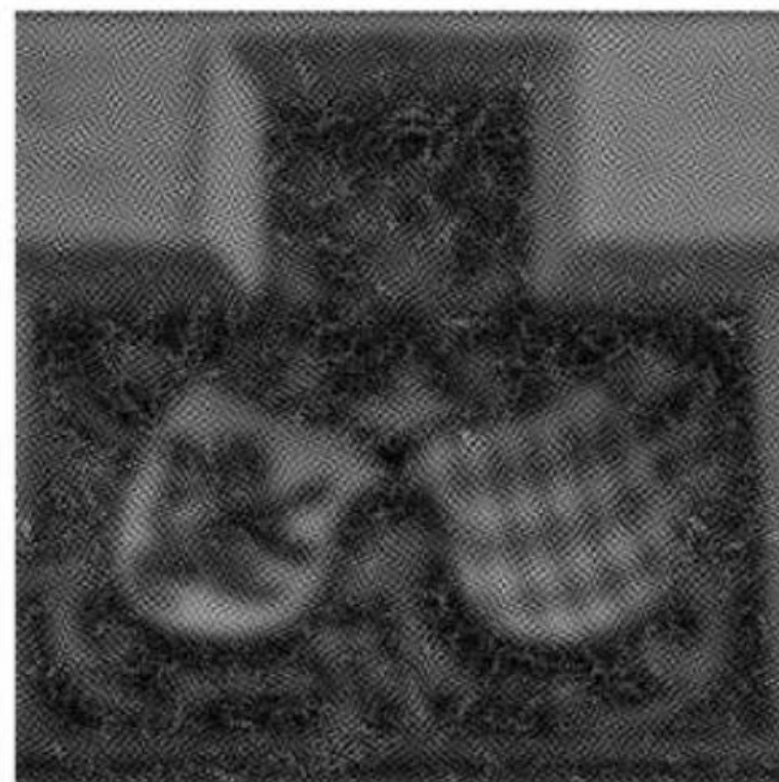
(d)

EXAMPLE 7.8

- FIGURE 12.11 Example of image restoration using inverse filtering: (e) results of using constrained division, with threshold $T = 0.01$; (f) same as (e), but with threshold $T = 0.001$.



(e)



(f)

EXAMPLE 7.8

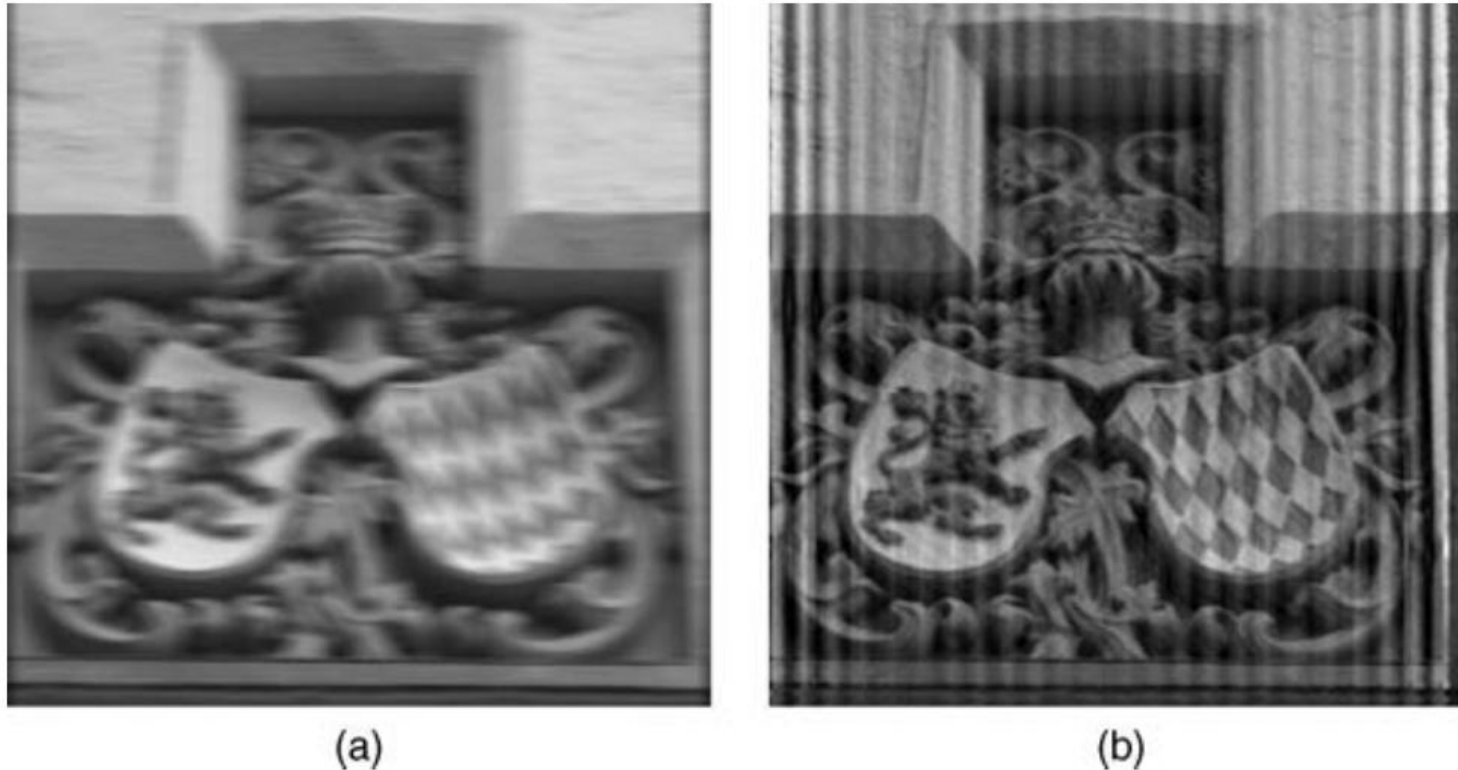


FIGURE 12.12 Example of motion deblurring using inverse filtering: (a) input image; (b) result of applying inverse filtering with constrained division and threshold $T = 0.05$: the motion blurred has been removed at the expense of the appearance of vertical artifacts.

7.6 WIENER FILTERING

- The Wiener filter (developed by Norbert Wiener in 1942) is an image restoration solution that can be applied to images that have been subject to a degradation function and also contain noise, which corresponds to the worst-case scenario for the degraded image $g(x, y)$ in Figure 12.1.
- The design of the Wiener filter is guided by an attempt to model the error in the restored image through statistical methods, particularly the minimum mean square *estimator*. once the error is modeled, the average error is mathematically minimized. Assuming a degraded version $g(x, y)$ of some original image $f(x, y)$, and a restored version $r(x, y)$, if we compute the sum of the squared differences between each and every pixel in $f(x, y)$ and the corresponding pixel in $r(x, y)$, we have a figure of merit that captures how well the restoration algorithm worked: smaller values mean better results.

7.6 WIENER FILTERING

- The transfer function of a Wiener filter is given by

$$R(u, v) = \left[\frac{1}{H(u, v)} \frac{|H(u, v)|^2}{|H(u, v)|^2 + K} \right] G(u, v)$$

where $H(u, v)$ is the degradation function and K is a constant used to approximate the amount of noise.

In MATLAB

- The IPT has a function that implements image deblurring using Wiener filter:
deconvwnr.

EXAMPLE 7.9

- Figure 12.13 shows an example of image restoration using Wiener filtering. Part (a) shows the input image, which has been degraded by blur and noise. Part (b) shows the results of applying inverse filtering, using a 10th-order Butterworth low-pass filter and limiting the restoration cutoff frequency to 50. Part (c) shows the results of using Wiener filter with different values of K , to illustrate the trade-off between noise reduction (which improves for higher values of K) and deblurring (which is best for lower values of K).
- The Wiener filter can also be used to restore images in the presence of blurring only (i.e., without noise), as shown in Figure 12.14. In such cases, the best results are obtained for lower values of K (Figure 12.14c), but the resulting image is still not as crisp as the one obtained with the inverse filter (Figure 12.14b).

EXAMPLE 7.9

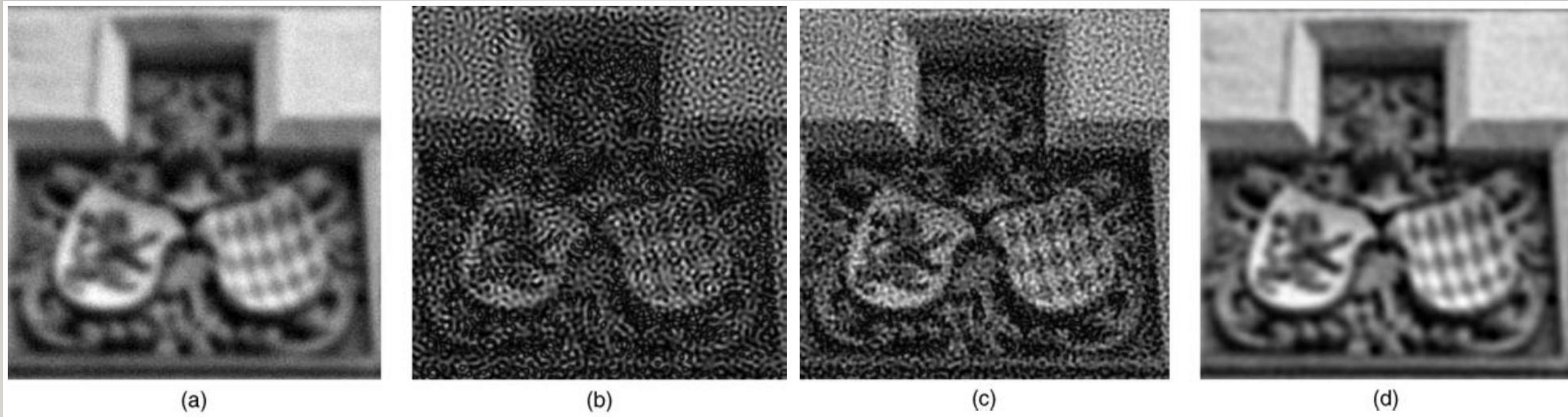


FIGURE 12.13 Example of image restoration using Wiener filtering: (a) input image (blurry and noisy); (b) result of inverse filtering, applying a 10th-order Butterworth low-pass filter with cutoff frequency of 50 to the division; (c) results of Wiener filter, with $K = 10^{-3}$; (d) same as (c), but with $K = 0.1$.

EXAMPLE 7.9

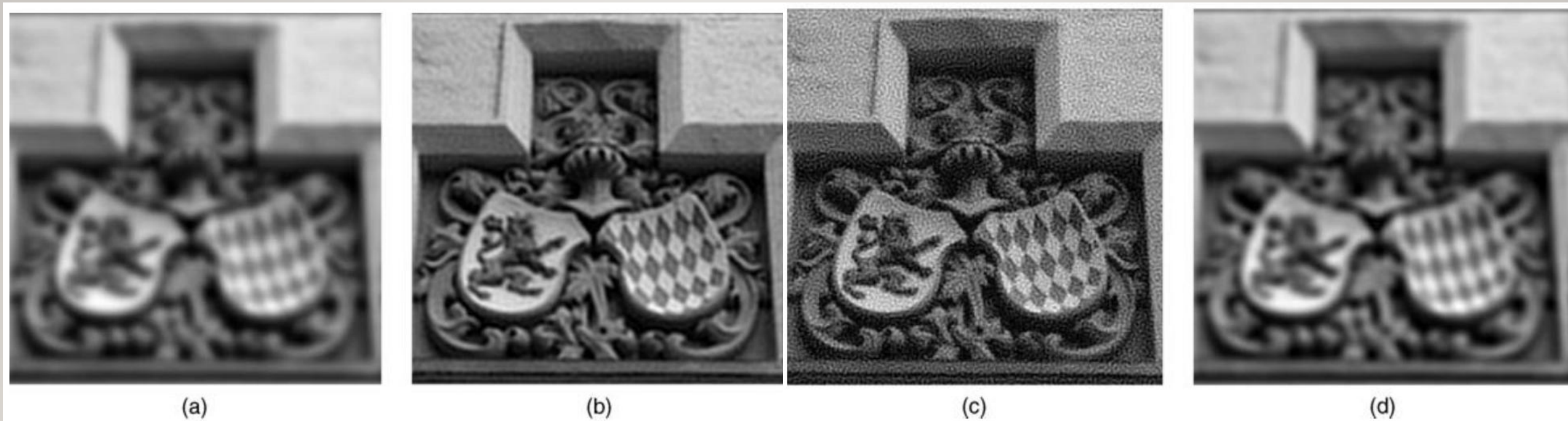


FIGURE 12.14 Example of image restoration using Wiener filtering: (a) input (blurry) image; (b) result of inverse filtering, applying a 10th-order Butterworth low-pass filter with cutoff frequency of 50 to the division; (c) results of Wiener filter, with $K = 10^{-5}$; (d) same as (c), but with $K = 0.1$.

7.6 WIENER FILTERING

In MATLAB

- The IPT has several other built-in functions for image deblurring, namely,
 - *deconvreg*. deblur image using regularized filter
 - *deconvlucy*. deblur image using Lucy–Richardson method
 - *deconvblind*. deblur image using blind deconvolution
- The complexity of these functions (in terms of both their mathematical formulation and the number of parameters that need to be adjusted for better performance) is beyond the scope of this book. The reader should refer to the corresponding MATLAB documentation and demos for additional information.

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Goal

- The goal of this tutorial is to learn how to perform noise reduction using spatial-domain techniques.

Objectives

- Learn how to implement the arithmetic mean filter, as well as some of its variations, such as the contraharmonic mean, the harmonic mean, and the geometric mean filters.
- Learn how to perform order statistic filtering, including median, min, max, mid-point, and alpha-trimmed mean filters.



7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Arithmetic Mean Filter

1. Load the *eight* image, add (Gaussian) noise to it, and display the image before and after adding noise.

```
I = imread('eight.tif');  
In = imnoise(I,'gaussian',0,0.001);  
figure, subplot(2,2,1), imshow(I), title('Original Image');  
subplot(2,2,2), imshow(In), title('Noisy Image');
```

2. Apply an averaging filter to the image using the default kernel size (3 x 3).

```
f1 = fspecial('average');  
I_blur1 = imfilter(In,f1);  
subplot(2,2,3), imshow(I_blur1), ...  
title('Averaging with default kernel size');
```

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Question 1: What is the general effect of the arithmetic mean filter?

3. Implement an averaging kernel with a 5 x 5 mask.

```
f2 = fspecial('average',[5 5]);  
I_blur2 = imfilter(In,f2);  
subplot(2,2,4), imshow(I_blur2), ...  
title('Averaging with 5x5 kernel');
```

Question 2: How does the size of the kernel affect the resulting image?



7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Contraharmonic Mean Filter

4. Close any open figures.

5. Load two noisy versions of the eight image: one with salt noise and the other with pepper. Also, display the original image along with the two affected images.

```
l_salt = im2double(imread('eight_salt.tif'));  
l_pepper = im2double(imread('eight_pepper.tif'));  
figure  
subplot(2,3,1), imshow(l), title('Original Image');  
subplot(2,3,2), imshow(l_salt), title('Salt Noise');  
subplot(2,3,3), imshow(l_pepper), title('Pepper Noise');
```

The contraharmonic function requires that images be of class double. This is why we convert the image when loading it.

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

6. Filter the salt noise affected image using -1 for the value of r .

```
l_fix1 = nlfiter(l_salt,[3 3],@c_harmonic,-1);  
subplot(2,3,5), imshow(l_fix1), title('Salt Removed, r = -1');
```

Our function *c_harmonic* takes two parameters: the current window matrix and a value for r . The window matrix, which gets stored into variable x , is passed implicitly by the *nlfiter* function.

Note how we specified the *c_harmonic* function as the third parameter of the *nlfiter* function call. When using the *nlfiter* function, if you want to pass any additional parameters to your function, you can specify those parameters after the function handle (labeled with a '@' in front of it). Notice above how we specified the value of r directly after the function handle.

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

7. Filter the pepper noise affected image using 1 for the value of r .

```
l_fix2 = nlfilter(l_pepper,[3 3],@c_harmonic,1);  
subplot(2,3,6), imshow(l_fix2), title('Pepper Removed, r = 1');
```

As mentioned previously, using the wrong sign for the value of r can lead to unwanted results.

8. Filter the pepper noise image using the wrong sign for r .

```
l_bad = nlfilter(l_pepper,[3 3],@c_harmonic,-1);  
subplot(2,3,4), imshow(l_bad), title('Using wrong sign for r');
```

Question 3: What is the effect of using the wrong sign when filtering with the contraharmonic mean filter?

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Harmonic Mean Filter

The harmonic mean filter is another variation of the mean filter and is good for salt and Gaussian noise. It fails, however, when used on pepper noise.

9. Close any open figures.

10. Filter the salt noise affected image with the harmonic filter.

```
l_fix4 = nlfilter(l_salt,[3 3],@harmonic);
```

```
figure
```

```
subplot(2,3,1), imshow(l), title('Original Image');
```

```
subplot(2,3,2), imshow(l_salt), title('Salt Noise');
```

```
subplot(2,3,3), imshow(l_pepper), title('Pepper Noise');
```

```
subplot(2,3,5), imshow(l_fix4), title('Harmonic Filtered (salt)');
```

11. Filter the pepper noise image and display the result.

```
l_bad2 = nlfilter(l_pepper,[3 3],@harmonic);
```

```
subplot(2,3,6), imshow(l_bad2), title('Harmonic Filtered (pepper)');
```


7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Question 4: Why does the harmonic mean filter fail for images with pepper noise?

12. Try to filter the `ln` image (I with additive Gaussian noise) with the harmonic mean filter. The image must be converted to double first.

```
ln_d = im2double(ln);  
l_fix5 = nlfilter(ln_d,[3 3],@harmonic);  
figure  
subplot(1,3,1), imshow(I), title('Original Image');  
subplot(1,3,2), imshow(ln_d), title('Image w/ Gaussian Noise');  
subplot(1,3,3), imshow(l_fix5), title('Filtered w/ Harmonic Mean');
```

Question 5: How does the size of the window affect the output image?

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Geometric Mean Filter

The last variation of the mean filters we will look at is the geometric mean filter. This filter is known to preserve image detail better than the arithmetic mean filter and works best on Gaussian noise.

13. Close any open figures.

14. Perform a geometric mean filter on the eight image with Gaussian noise (currently loaded in the variable `ln_d`).

```
l_fix6 = nlfilter(ln_d,[3 3],@geometric);  
figure  
subplot(1,3,1), imshow(l), title('Original Image');  
subplot(1,3,2), imshow(ln_d), title('Gaussian Noise');  
subplot(1,3,3), imshow(l_fix6), title('Geometric Mean Filtered');
```

Question 6: Filter the salt and pepper noise images with the geometric mean filter. How does the filter perform?

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Order Statistic Filters

The median filter is the most popular example of an order statistic filter. This filter simply sorts all values within a window, finds the median value, and replaces the original pixel value with the median value. It is commonly used for salt and pepper noise. Because of its popularity, the median filter has its own function (*medfilt2*) provided by the IPT.

15. Close any open figures and clear all workspace variables.

16. Load the coins image and apply salt and pepper noise.



7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

```
I = imread('coins.png');  
I_snp = imnoise(I, 'salt & pepper');  
figure  
subplot(1,3,1), imshow(I), title('Original Image');  
subplot(1,3,2), imshow(I_snp), title('Salt & Pepper Noise');
```

17. Filter the image using the medfilt2 function.

```
I_filt = medfilt2(I_snp, [3 3]);  
subplot(1,3,3), imshow(I_filt), title('Filtered Image');
```

Question 7: How does the size of the window affect the output image?

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

18. Apply the filter to an image with Gaussian noise.

```
I_g = imnoise(I,'gaussian');  
I_filt2 = medfilt2(I_g,[3 3]);  
figure  
subplot(1,3,1), imshow(I), title('Original Image');  
subplot(1,3,2), imshow(I_g), title('Gaussian Noise');  
subplot(1,3,3), imshow(I_filt2), title('Filtered');
```

Question 8: Why do you think the median filter works on salt and pepper noise but not Gaussian noise?

A quickway to get rid of salt noise in an image is to use the min filter, which simply takes the minimum value of a window when the values are ordered. Recall that we previously used the *imfilter* function when dealing with convolution masks and *nlfilter* for sliding neighborhood operations that could not be implemented as a convolution mask. Similarly, the *ordfilt2* function is used for order statistic operations.

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

19. Close any open figures and clear all workspace variables.

20. Use the `ordfilt2` function to implement a min filter on an image with salt noise.

```
I_s = imread('eight_salt.tif');  
I2 = ordfilt2(I_s, 1, ones(3,3));  
figure  
subplot(1,2,1), imshow(I_s), title('Salt Noise');  
subplot(1,2,2), imshow(I2), title('Min Filter');
```


7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Question 9: Why would this filter not work on pepper noise?

The first parameter specified in the `ordfilt2` function is the image we wish to filter. The second parameter specifies the index of the value to be used after all values in the window have been ordered. Here, we specified this parameter as 1, which means we want the first value after reordering, that is, the minimum value.

The last parameter defines the size of the window as well as which values in that window will be used in the ordering.



7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Question 9: Why would this filter not work on pepper noise?

A 3×3 matrix of 1's would indicate a 3×3 window and to use all values when ordering. If we instead specified a 3×3 matrix where only the first row was 1's and the last two rows were zeros, then the sliding window would consist of a 3×3 matrix, but only the top three values would be considered when ordering.

In addition, keep in mind that even though we used a special function to implement the median filter, it is still an order statistic filter, which means we could have implemented it using the `ordfilt2` function.



7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Question 10: Implement the median filter using the `ordfilt2` function.

The max filter is used for filtering pepper noise, similar to the technique of the min filter.

21. Filter a pepper noise affected image with the max filter.

```
I_p = imread('eight_pepper.tif');  
I3 = ordfilt2(I_p, 9, ones(3,3));  
figure  
subplot(1,2,1), imshow(I_p), title('Pepper Noise');  
subplot(1,2,2), imshow(I3), title('Max Filter');
```


7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

22. Filter an image contaminated with Gaussian noise using the midpoint filter.

```
I = imread('coins.png');  
I_g = imnoise(I,'gaussian',0,0.001);  
midpoint = inline('0.5 * (max(x(:)) + min(x(:)))');  
I_filt = nlfilter(I_g,[3 3],midpoint);  
figure  
subplot(1,2,1), imshow(I_g), title('Gaussian Noise');  
subplot(1,2,2), imshow(I_filt), title('Midpoint Filter');
```

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

Alpha-Trimmed Mean Filters

The alpha-trimmed mean filter is basically an averaging filter whose outlying values are removed before averaging. To do this, we sort the values in the window, discard elements on both ends, and then take the average of the remaining values. This has been defined in the function *atmean*.

23. Close any open figures and clear all workspace variables.

24. Generate a noisy image with Gaussian noise and salt and pepper noise.

```
I = imread('cameraman.tif');  
Id = im2double(I);  
In = imnoise(Id, 'salt & pepper');  
In2 = imnoise(In, 'gaussian');
```

7.7 TUTORIAL: NOISE REDUCTION USING SPATIAL-DOMAIN TECHNIQUES

25. Filter the image using the alpha-trimmed mean filter.

```
I_filt = nlfilter(In2,[5 5],@atmean,6);
```

figure

```
subplot(1,3,1), imshow(I), title('Original Image');
```

```
subplot(1,3,2), imshow(In2), title('S&P and Gaussian Noise');
```

```
subplot(1,3,3), imshow(I_filt), title('Alpha Trimmed Mean');
```

Question 11: When filtering an image with both types of noise, how does the alpha-trimmed mean filter compare to the arithmetic mean filter?

WHAT HAVE WE LEARNED?

- In the context of image processing, noise is a general term used to express deviations from a pixel's expected (or true) value. When these deviations are offsets from the true value, the noise is said to be of additive type. When the true value is rescaled as a result of noise, such noise is said to be of multiplicative type.
- The statistical properties of noise are usually modeled in a way that is independent of the actual causes of the noise. Common probability distribution functions associated with noise are Gaussian, exponential, uniform, and gamma (Erlang).

WHAT HAVE WE LEARNED?

- Two of the most common types of noise in image processing are the Gaussian noise and the salt and pepper noise. Both are forms of additive noise. The Gaussian noise follows a zero-mean normal distribution. The salt and pepper noise is a type of impulsive noise that appears as black and white specks on the image.
- The most common noise removal techniques in the spatial domain are the mean filter, the median filter, and variants and combinations of them.
- The most common noise removal techniques in the frequency domain are low-pass, bandpass, bandreject, and notch filters.



WHAT HAVE WE LEARNED?

- *Blurring* is the loss of sharpness in an image. It can be caused by poor focusing, relative motion between sensor and scene, and noise, among other factors.
- Deblurring techniques typically consist of applying inverse filtering techniques with the goal of “undoing” the degradation. The best-known approach to image deblurring (even in the presence of noise) is the Wiener filter, which is implemented in MATLAB by the *deconvwnr* function.

PROBLEMS

Problem 1. Write a modified and expanded version of the IPT function *imnoise*. Your function should allow the specification of other types of noise, currently not supported by *imnoise*, for example, Rayleigh, Erlang, uniform, and exponential.

Problem 2. What is the effect on an image if we apply a contraharmonic mean filter with the following values of R :

(a) $R = 0$

(b) $R = -1$

Problem 3. Write a MATLAB function to implement an ideal bandreject filter of radius D_0 and width W and use it to reduce periodic noise on an input noisy image.

PROBLEMS

Problem 4. Write a MATLAB function to implement a Butterworth bandreject filter of order n , radius D_0 , and width W and use it to reduce periodic noise on an input noisy image.

Problem 5. Write a MATLAB function to implement an ideal bandpass filter of radius D_0 and width W .

Problem 6. Write a MATLAB function to implement a Butterworth bandpass filter of order n , radius D_0 , and width W .

Problem 7. Write a MATLAB function to implement a Gaussian notch reject filter of radius D_0 , centered at (u_0, v_0) , and use it to reduce periodic noise on an input noisy image.

PROBLEMS

Problem 8. Write a MATLAB function to implement a *range filter* whose output is the difference between the maximum and the minimum gray levels in a neighborhood centered on a pixel [Eff00], test it, and answer the following questions:

- (a) Is the range filter a linear or nonlinear one? Explain.
- (b) What can it be used for?



END OF LECTURE 7

**LECTURE 8 – MORPHOLOGICAL IMAGE
PROCESSING**

